TRƯỜNG ĐẠI HỌC QUY NHƠN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP CHƯƠNG 6

Họ và tên: ..... MSSV: .....

MẪU THIẾT KẾ COMMAND
THỰC HIỆN UNDO VÀ REDO
Bài 6.15 đến 6.18 | Báo cáo cá nhân

....., 2026

Phạm vi báo cáo

Trong các bài 6.15–6.18, em trình bày cách mở rộng chương trình quản lý lịch biểu diễn bằng mẫu Command. Hướng giải quyết là lưu trạng thái trước mỗi thao tác, quản lý lịch sử bằng stack và đưa chức năng undo, redo vào Program. Các ví dụ dưới đây minh họa phương pháp và kết quả dự kiến.

1 Bài 6.15 Bổ sung undo cho command

Yêu cầu là hoàn tác các thao tác thêm show, xóa show và xóa toàn bộ lịch của bài 6.14. Em bổ sung undo() vào interface Command, giữ execute() và tạo các command bằng anonymous class trong Commands.

Trước khi execute(), command add và remove lưu show cũ của ngày được chọn qua Program.get(). Command clear lưu đủ bảy ngày vào EnumMap<Day, Show>. Khi undo(), dữ liệu đã lưu được đưa trở lại lịch. Việc sao chép Show giúp trạng thái cũ không bị dùng chung ngoài ý muốn.

```
    add1.execute(); add2.execute();
remove.execute(); clear.execute();
clear.undo(); remove.undo();
add2.undo(); add1.undo();
```

Ví dụ, thêm Movie 1 vào thứ Hai, thêm Movie 2 vào thứ Ba, xóa thứ Hai rồi xóa toàn bộ lịch. Undo clear dự kiến khôi phục Movie 2; undo remove khôi phục thêm Movie 1. Hai lần undo tiếp theo đưa lịch về trạng thái ban đầu. Cần lưu và so sánh lịch sau từng bước để phát hiện lỗi, thay vì chỉ kiểm tra chương trình không báo lỗi.

Nếu chỉ xóa dữ liệu mà không lưu trạng thái trước đó, chương trình không biết show nào cần khôi phục. Với một vài thao tác, client có thể tự giữ dữ liệu cũ, nhưng cách này dễ lặp code khi bổ sung chức năng.

2 Bài 6.16 Quản lý lịch sử bằng CommandProcessor

Em tạo CommandProcessor với một Deque<Command> dùng như stack. execute(command) gọi command.execute() rồi push command vào stack. undoLast() lấy command gần nhất, gọi undo() và chỉ bỏ nó khỏi lịch sử sau khi hoàn tác thành công. Nếu stack rỗng thì không thực hiện gì.

```
    processor.execute(Commands.add(program, movie, Day.MONDAY));
processor.undoLast();
```

Client chỉ yêu cầu processor thực thi hoặc hoàn tác, không lấy stack ra để tự xử lý. Cách này tuân thủ Law of Demeter vì client không cần biết cấu trúc lịch sử bên trong. Nếu chưa có processor, client phải tự nhớ các command và gọi undo theo đúng thứ tự ngược như bài 6.15.

3 Bài 6.17 Bổ sung redo

Em thêm redoStack để chứa các command đã được hoàn tác. Khi undo thành công, command được chuyển từ undoStack sang redoStack. Khi redo, processor gọi lại execute() của command ở đầu redoStack rồi chuyển command về undoStack.

```
    processor.undoLast(); // Chuyen command sang redoStack
processor.redoLast(); // Thuc hien lai command vua undo
```

Ví dụ, sau khi thêm Movie 1 vào thứ Hai, undo dự kiến đưa ngày này về trạng thái cũ; redo thêm Movie 1 trở lại. Nếu người dùng undo rồi thêm Movie 3 vào thứ Tư, execute() phải xóa redoStack vì lịch đã chuyển sang một nhánh thao tác mới.

Không gọi processor.execute() bên trong redoLast(), vì hàm đó xóa toàn bộ redoStack và làm mất các thao tác còn có thể redo. Với stack rỗng, redoLast() không thay đổi lịch. Nếu chỉ giữ undoStack, chương trình có thể hoàn tác nhưng không còn command để thực hiện lại.

4 Bài 6.18 Đóng gói processor trong Program

Em đặt một CommandProcessor ở bên trong Program. Các hàm public add(), remove() và clear() tạo command rồi nhờ processor thực thi. Hai hàm undoLast() và redoLast() cũng ủy quyền cho processor. Nhờ đó, client thao tác trực tiếp với Program mà không phải tạo command.

```
    Program program = new Program();
program.add(new Movie("Movie 1", 2017, 120), Day.MONDAY);
program.remove(Day.MONDAY);
program.undoLast(); // Du kien khoi phuc Movie 1
program.redoLast(); // Du kien xoa Movie 1 lan nua
```

Để tránh gọi lặp vô hạn, em tách addDirect(), removeDirect() và clearDirect() thành các hàm nội bộ không ghi lịch sử. Command gọi các hàm này trong execute() và undo(), không gọi lại các hàm public. Constructor cũng dùng clearDirect() để việc khởi tạo lịch không trở thành một thao tác có thể undo.

Program chịu trách nhiệm quản lý lịch; CommandProcessor quản lý hai stack; mỗi command lưu trạng thái cần khôi phục. Việc chia trách nhiệm giúp Program không trở thành God class. Nếu Program tự quản lý cả dữ liệu lẫn hai stack, lớp sẽ khó đọc và khó sửa hơn khi lịch sử thao tác thay đổi.

5 Nhận xét

Qua bốn bài, em hiểu rằng undo cần dữ liệu cũ, còn undo nhiều bước cần một nơi quản lý thứ tự thao tác. Hai stack giúp bổ sung redo và việc ủy quyền giữ cho Program gọn hơn. Khi kiểm tra, cần xét thêm lịch sử rỗng, ghi đè show đã có và thao tác mới sau undo. Cách này lưu lịch sử trong bộ nhớ nên chưa duy trì được lịch sử sau khi đóng chương trình.

Tài liệu tham khảo: Martin P. Robillard, DesignBook, Chapter 6 Practice Exercises, bài 15–18.
https://github.com/prmr/DesignBook/blob/master/exercises/e-chapter6.md